Lab 5

Process Synchronization and Message Passing in XINU

1. Introduction/Description

This lab introduces students to **XINU OS** and its mechanisms for process creation, inter-process communication (IPC), and synchronization. Students will learn how to install and run XINU using WSL or Ubuntu, execute simple processes, and implement basic IPC using `send()` and `receive()`. The lab also covers synchronization techniques including semaphores, sleep-based waits, and handling multiple senders communicating with a single receiver. These exercises provide hands-on experience with fundamental operating system concepts in a minimal, educational kernel environment.

2. Required Equipment / Software

Computer System

- Any Linux-based environment (Ubuntu, Debian, Fedora, Arch, etc.)
- Alternatively, WSL2 with Ubuntu on Windows

Terminal Access

- Bash shell (default on most Linux systems)
- Basic text editor: nano, vim, or VS Code terminal

Software Utilities

- Standard GNU tools:
  - bash, coreutils, grep, find, bc, printf
- Optional but useful:
  - tree, man-db
- XINU-specific tools:
  - QEMU (`qemu-system-x86`, `qemu-utils`)
  - Flex, Bison, Gawk
  - GRUB tools (`grub-common`, `grub-pc-bin`, `xorriso`)

Script / Code Files

- C files (`.c`) for XINU processes created during the experiment

Internet Access (optional)

- For cloning XINU repository, checking manuals, or troubleshooting

3. Background Knowledge

Install WSL (Skip if you are already using Ubuntu)

wsl --install

X Server (Optional. Only needed for GUI.)

download and install https://sourceforge.net/projects/vcxsrv/
Start the X server application.

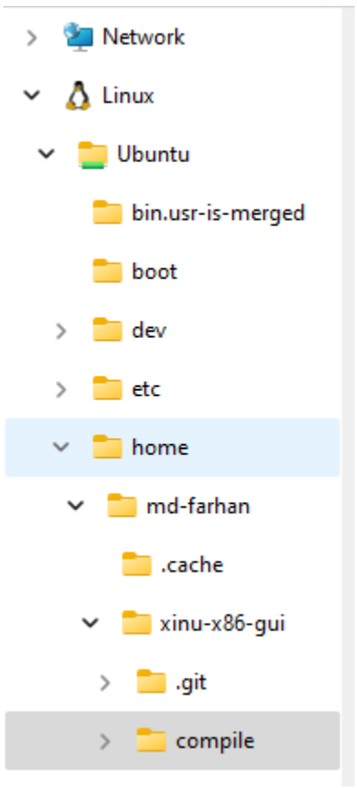
Install QEMU

sudo apt update
sudo apt upgrade
sudo apt install qemu-system qemu-utils
sudo apt install flex bison gawk qemu-system-x86 xorriso grub-common grub-pc-bin

Build XINU OS

git clone https://github.com/zrafa/xinu-x86-gui

Navigate to the Makefile using file explorer.
For WSL.
Linux > Ubuntu > home > username >xinu-x86-gui > compile
For Ubuntu
home > username >xinu-x86-gui > compile



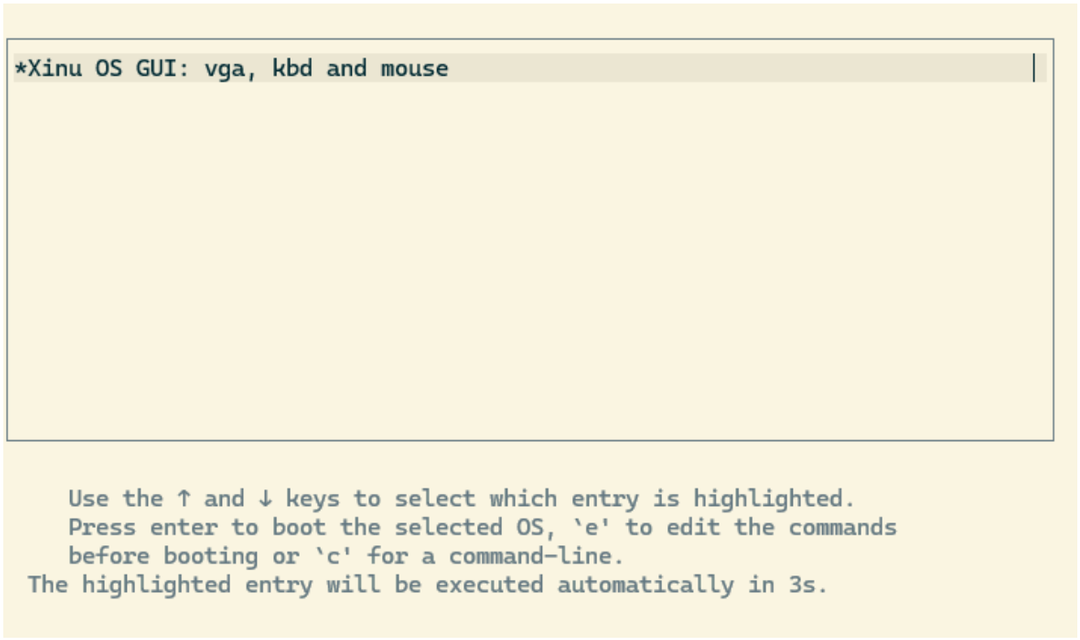
Inside Makefile, find run-qemu and add this to the end of the command -nographic

```
172  # Generates the Grub2-based ISO - see bochsrc.txt for info.
173  # Requires grub2-mkrescue and xorriso be installed!
174  run-qemu: qemu-iso-pre
175      echo OTRO GRUB PARA PROBAR: grub-mkrescue -o disk.iso test/img
176      grub-mkrescue -d /usr/lib/grub/i386-pc -o disk.iso test/img
177      qemu-system-i386 -machine accel=kvm -enable-kvm -drive
         file=disk.iso,index=0,media=cdrom -netdev user,id=u0 -device
         e1000-82545em,netdev=u0 -nographic
```

(run once for every time you boot WSL/Ubuntu)
sudo chmod 666 /dev/kvm

(run every time you boot XINU)
cd xinu-x86-gui/compile
make clean

make
make run-qemu



Shut down XINU

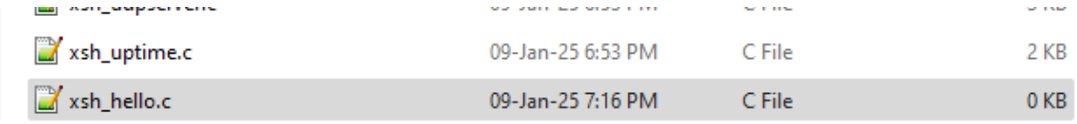
Open another terminal. (For WSL, you need to start a new WSL terminal.)
Use the top command to see running processes. Get PID of QEMU.
Press q to exit top.
kill <PID>

```
md-farhan@DESKTOP-SJGL3N:~$ top
top - 19:11:52 up  1:19,  1 user,  load average: 0.93, 1.22, 0.98
Tasks:  32 total,   1 running,  31 sleeping,   0 stopped,   0 zombie
%Cpu(s): 33.3 us, 42.4 sy,  0.0 ni, 15.2 id,  3.0 wa,  0.0 hi,  6.1 si,  0.0 st
MiB Mem :   1875.0 total,     75.5 free,    540.4 used,   1438.3 buff/cache
MiB Swap:   1024.0 total,   1021.7 free,      2.3 used.   1334.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  12799 md-farh+  20   0  542432  72200  28956 S  76.9   3.8   0:16.52 qemu-system-i38
      1 root      20   0   22228  13292   9520 S   0.0   0.7   0:09.00 systemd
      2 root      20   0    2616   1420   1320 S   0.0   0.1   0:00.07 init-systemd(Ub
      6 root      20   0    2644    132    132 S   0.0   0.0   0:01.41 init
     60 root      19  -1   66832  15780  14528 S   0.0   0.8   1:07.32 systemd-journal
    107 root      20   0   24092   6084   4788 S   0.0   0.3   0:03.79 systemd-udevd
    118 systemd+  20   0   21452  11696   9504 S   0.0   0.6   0:00.37 systemd-resolve
    119 systemd+  20   0   91020   6460   5612 S   0.0   0.3   0:00.56 systemd-timesyn
    159 root      20   0    4236   2668   2444 S   0.0   0.1   0:00.02 cron
    160 message+  20   0    9756   5324   4372 S   0.0   0.3   0:01.94 dbus-daemon
    170 root      20   0   17976   8352   7332 S   0.0   0.4   0:00.84 systemd-logind
    173 root      20   0 1681852  11256   4768 S   0.0   0.6   0:02.01 wsl-pro-service
    182 root      20   0    3160   1072    988 S   0.0   0.1   0:00.02 agetty
    189 root      20   0    3116   1076    988 S   0.0   0.1   0:00.02 agetty
    196 root      20   0  107000  22024  12696 S   0.0   1.1   0:00.31 unattended-upgr
    288 root      20   0    2624     96      0 S   0.0   0.0   0:00.00 SessionLeader
    289 root      20   0    2624    112      0 S   0.0   0.0   0:02.66 Relay(292)
    292 md-farh+  20   0    6072   4464   3036 S   0.0   0.2   0:00.70 bash
    293 root      20   0    6696   3888   3276 S   0.0   0.2   0:00.02 login
    382 md-farh+  20   0   20256  11276   9196 S   0.0   0.6   0:00.68 systemd
    383 md-farh+  20   0   21148    880      0 S   0.0   0.0   0:00.00 (sd-pam)
    394 md-farh+  20   0    6056   3500   2000 S   0.0   0.2   0:00.03 bash
    959 polkitd   20   0  308160   6768   6120 S   0.0   0.4   0:00.93 polkitd
md-farhan@DESKTOP-SJGL3NT:~$ kill 12799
```

Running code in XINU

Create a c file, xsh_hello.c, in xinu-x86-gui > shell



Open and write code inside it

```c
#include <xinu.h>
#include <stdio.h>

shellcmd xsh_hello(int nargs, char *args[]) {
    printf("Hello World\n");
    return 0;
}

```

Open cmdtab.c from same directory (xinu-x86-gui > shell)

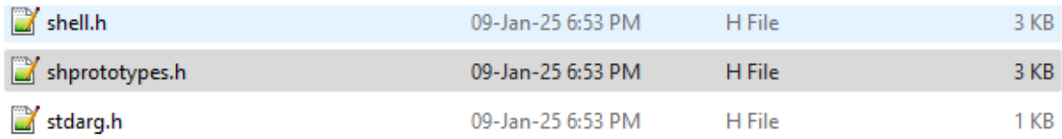
Add new entry

```c
#include <xinu.h>
#include "shprototypes.h"

/*****************************************************
/* Table of Xinu shell commands and the function ass
/*****************************************************
const   struct  cmdent  cmdtab[] = {
    {"hello", FALSE, xsh_hello},
    {"argecho", TRUE,   xsh_argecho},
    {"arp",     FALSE,  xsh_arp},
    {"cat",     FALSE,  xsh_cat},
```

(FALSE, because hello is not a built in command)

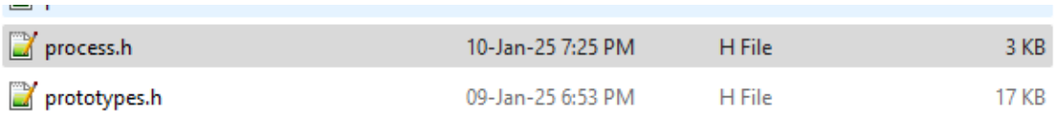
Open shprototypes.h header file from xinu-x86-gui > include



Add prototype of our function

```
/* in file xsh_hello.c */
extern  shellcmd  xsh_hello (int32, char *[]);
```

Open process.h header file from xinu-x86-gui > include



Change NPROC from 8 to 20

```
#ifndef NPROC
#define NPROC       20
#endif
```

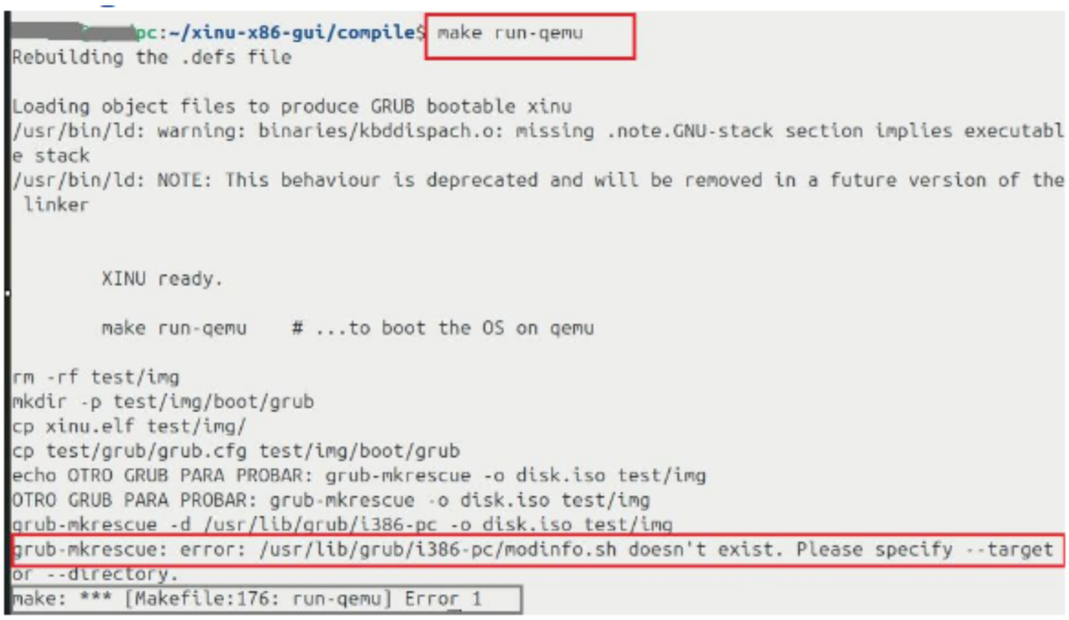
Build and run the the OS from xinu-x86-gui > compile directory
cd xinu-x86-gui/compile
make clean
make
make run-qemu

In xinu shell, type command hello and press enter

```
xsh $
xsh $
xsh $
xsh $ hello
Hello World
xsh $
```

Troubleshooting

This section was contributed by Syed Shifat E Rahman (2003065)
You might encounter grub related errors while running make run-qemu.



sudo apt install grub-pc-bin
sudo apt reinstall grub-pc-bin
grub-mkrescue --target=i386-pc -d /usr/lib/grub/i386-pc -o disk.iso test/img
make run-qemu

Simple Message Passing

Concept: One process sends a message to another using XINU's send() and receive() system calls.

Example (from XINU book, slightly simplified):

```
#include <xinu.h>

process sender(pid32 pid) {
    char msg = 'A';
    kprintf("Sender: sending message '%c' to process %d\n", msg, pid);
    send(pid, msg);
    return OK;
}

process receiver() {
    char msg;
    msg = receive();
    kprintf("Receiver: got message '%c'\n", msg);
    return OK;
}

process main() {
    pid32 rpid, spid;

    rpid = create(receiver, 1024, 20, "receiver", 0);
    spid = create(sender, 1024, 20, "sender", 1, rpid);

    resume(rpid);
    resume(spid);

    return OK;
}
```

Producer-Consumer with Semaphores

Concept: Classic synchronization problem using XINU semaphores.

```
#include <xinu.h>

sid32 mutex, items, spaces;
int buffer[5];
int in = 0, out = 0;

process producer() {
    int i;
    for (i = 0; i < 10; i++) {
        wait(spaces);       // Wait for empty slot
        wait(mutex);        // Enter critical section

        buffer[in] = i;
        kprintf("Produced %d\n", i);
        in = (in + 1) % 5;

        signal(mutex);      // Exit critical section
        signal(items);      // Increment items count
    }
    return OK;
}
```

```
process consumer() {
    int i, val;
    for (i = 0; i < 10; i++) {
        wait(items);        // Wait for items
        wait(mutex);        // Enter critical section

        val = buffer[out];
        kprintf("Consumed %d\n", val);
        out = (out + 1) % 5;

        signal(mutex);      // Exit critical section
        signal(spaces);     // Increment space count
    }
    return OK;
}

process main() {
    mutex = semcreate(1);
    items = semcreate(0);
    spaces = semcreate(5);

    resume(create(producer, 1024, 20, "producer", 0));
    resume(create(consumer, 1024, 20, "consumer", 0));

    return OK;
}
```

Using Sleep for Synchronization

Concept: Synchronization without semaphores using sleepms() for timed waits.

```
#include <xinu.h>

process child() {
    kprintf("Child process starting, will sleep 2 seconds\n");
    sleepms(2000);
    kprintf("Child process awake\n");
    return OK;
}

process main() {
    pid32 pid;
    pid = create(child, 1024, 20, "child", 0);
    resume(pid);

    kprintf("Parent waiting 3 seconds\n");
    sleepms(3000);
    kprintf("Parent done waiting\n");
    return OK;
}
```

Multiple Senders and Single Receiver

Concept: Multiple processes send messages to one receiver process. XINU handles queuing of messages automatically.

```
#include <xinu.h>

process sender(pid32 pid, char msg) {
    kprintf("Sender sending '%c'\n", msg);
    send(pid, msg);
    return OK;
}

process receiver() {
    char msg;
    int i;
    for (i = 0; i < 3; i++) {
        msg = receive();
        kprintf("Receiver got '%c'\n", msg);
    }
    return OK;
}

process main() {
    pid32 rpid = create(receiver, 1024, 20, "receiver", 0);
    resume(rpid);

    resume(create(sender, 1024, 20, "sender1", 2, rpid, 'A'));
    resume(create(sender, 1024, 20, "sender2", 2, rpid, 'B'));
    resume(create(sender, 1024, 20, "sender3", 2, rpid, 'C'));

    return OK;
}
```

4. Precautions

- Always check return values of system calls like `create()`, `send()`, `receive()`, and `semcreate()`.
- Make sure to set appropriate permissions for `/dev/kvm` (`sudo chmod 666 /dev/kvm`) to avoid QEMU errors.
- Close unused pipe descriptors and clean previous builds (`make clean`) before running the OS to prevent errors.
- When modifying `NPROC` in `process.h`, ensure it matches the number of expected concurrent processes to avoid undefined behavior.
- When testing multiple processes, ensure that console output is readable; consider using `kprintf()` in moderation to avoid excessive interleaving.
- Always use `sleepms()` carefully; timed waits may cause unexpected results if misused.
- Restart WSL/Ubuntu if QEMU or XINU becomes unresponsive.

5. Conclusion / Discussion

XINU OS Overview: Students observe how a minimal operating system manages processes, scheduling, and memory.

Process Creation and Management: The `create()` system call is used to allocate process memory and set priorities. `resume()` activates the process, demonstrating cooperative scheduling.

Message Passing:

- `send()` and `receive()` enable one-to-one communication between processes.

- XINU handles message queuing automatically, allowing safe communication without shared memory.

Semaphores for Synchronization:

- `semcreate()`, `wait()`, and `signal()` implement mutual exclusion and coordination between producer and consumer processes.
- The producer-consumer problem highlights critical section management and avoiding race conditions.

Sleep-based Synchronization: Timed waits using `sleepms()` provide a simple mechanism for delaying process execution without semaphores, useful for simulating workload or ordering processes.

Multiple Senders / Single Receiver: XINU queues multiple messages for a single receiving process, demonstrating its handling of concurrent IPC in a simple but deterministic way.

Practical Learning Outcomes:

- Understanding kernel-level process management and scheduling
- Implementing safe IPC mechanisms
- Synchronizing multiple processes with semaphores or timed waits
- Debugging and building an educational OS from source using QEMU

Exercise

1. Create a **multi-stage pipeline** in XINU where one process reads integers from an array, a second filters out even numbers, a third computes running sums, and a fourth prints results. Synchronize stages using semaphores to prevent data races and ensure no stage overwrites data before the next consumes it.
2. Implement a **multi-client, single-server system** where several client processes send strings to a server. The server appends a timestamp and sends acknowledgments back. Handle blocked sends when the server queue is full and ensure messages are processed in order without loss.
3. Design a **multi-producer, multi-consumer buffer** where producers generate random numbers and consumers compute factorials. Use semaphores for mutual exclusion and to track empty and full slots, ensuring no consumer starvation even if producers are faster.
4. Create a **dynamic process synchronization exercise** where a parent spawns children with varying task durations. Children send completion messages to the parent, which counts messages and prints status updates, demonstrating asynchronous execution and robust parent synchronization without using wait().
5. Develop a **priority-based IPC system** with high, medium, and low priority processes sending messages to a central server. The server must process high-priority messages first while still preventing starvation of lower-priority processes, using semaphores or XINU queues for synchronization.